

ROBOTICS · LESSON 6

The Motor Checks Itself

An encoder measures the motor's real speed and a PID loop corrects the command until it matches — the motor checks whether it actually did what it was told.

ROBO-006 · ~50 MIN · INTERMEDIATE

1. The one idea

Feedback. The motor measures its **own** speed with an encoder, compares it to the speed you asked for, and a **PID** loop nudges the command up or down until the two match. The motor checks whether it actually did what it was told — and fixes itself when it didn't.

2. Why it matters

Open-loop PWM is a guess. "200 of 255" is one speed on a fresh battery and a slower one on a flat battery, faster downhill than up, and different on motor A than motor B — which is exactly why the Lesson 4 rover veered. Closed-loop control measures the real outcome and corrects toward the target. PID is the workhorse of all control — rovers, drones, thermostats, cruise control. This is the control-theory lesson Lesson 1 deliberately skipped and Lessons 4–5 kept pointing at. You've earned it.

3. Parts & wiring

A motor with a **quadrature encoder** (many gear motors ship with a 6-wire hall encoder: motor +/–, encoder VCC/GND, channel A, channel B). One motor is enough — keep the loop simple.

Wire	To	Why
Motor +/–	L298N OUT1/OUT2	driven as in Lesson 4 (IN1=25, IN2=26, ENA=27)
Encoder VCC / GND	3V3 / GND	powers the hall sensors
Encoder A	GPIO18	counted on an interrupt
Encoder B	GPIO19	read in the interrupt to get direction

4. Predict (do this before uploading)

- With only the **P** term (a small K_p), will the motor ever quite reach the target speed, or settle just short?
- If you add an **I** term, what happens to that gap over a second or two?
- If you crank K_p way up, what does the speed do — hold steady, or hunt around the target?

5. Build it — the command comes from the error

```
const int IN1 = 25, IN2 = 26, ENA = 27;    // one motor (Lesson 4 wiring)
const int ENC_A = 18, ENC_B = 19;         // its quadrature encoder

volatile long ticks = 0;

void IRAM_ATTR onEncA() {                  // tiny ISR: count, with direction
    ticks += digitalRead(ENC_B) ? +1 : -1;
}

float Kp = 0.6, Ki = 1.5, Kd = 0.02;      // tune these
float integral = 0, prevError = 0;
long lastTicks = 0;
unsigned long lastMs = 0;
float targetTPS = 600;                    // target speed, encoder ticks/second

void setup() {
    Serial.begin(115200);
    pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT); pinMode(ENA, OUTPUT);
    pinMode(ENC_A, INPUT_PULLUP); pinMode(ENC_B, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(ENC_A), onEncA, RISING);
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);    // forward
}

void loop() {
    unsigned long now = millis();
    if (now - lastMs < 50) return;           // run the loop at 20 Hz
    float dt = (now - lastMs) / 1000.0;
    lastMs = now;

    long d = ticks - lastTicks; lastTicks = ticks;
    float measured = d / dt;                 // ticks per second, right now

    float error = targetTPS - measured;      // how wrong are we?
    integral += error * dt;
    float deriv = (error - prevError) / dt;
    prevError = error;

    float out = Kp * error + Ki * integral + Kd * deriv;
```

```

    out = constrain(out, 0, 255);
    analogWrite(ENA, out);

    Serial.printf("target=%.0f  measured=%.0f  out=%.0f\n", targetTPS, measured, out);
}

```

The new idea: **you no longer set the command.** `out` is computed every loop from the *error* between what you asked and what the motor did. `Kp` reacts to the error now, `Ki` erases the small gap that lingers, `Kd` damps the overshoot. P is the heart; I and D are the polish.

6. Test against your prediction

Open the Serial Monitor at 115200 baud and watch `measured` climb to `target` and hold. Now pinch the wheel with your finger — `out` shoots up as the loop fights to keep the speed, and eases back when you let go. The motor *defends* its target against your hand. That's feedback you can feel.

7. What goes wrong (pre-loaded debugging)

- `measured` stays 0 → encoder unpowered, A/B not on the pins you attached the interrupt to, or no `INPUT_PULLUP`.
- **Motor pins full speed and won't regulate** → a sign error: your `error` (or the encoder's count direction) is backwards, so the loop pushes the wrong way. Flip the error sign or swap A/B.
- **Speed hunts up and down around the target** → `Kp` too high. Lower it, add a little `Kd`.
- **Settles just below target and stays there** → that's steady-state error — pure P can't close it. Raise `Ki`.
- **Overshoots wildly after a stall** → integral windup. Clamp `integral` to a sane range.
- **ESP32 resets** → too much in the ISR. Keep it `IRAM_ATTR`, touch only `volatile` s, never `Serial` / `delay` inside it.

8. Checkpoint

The motor now holds a commanded speed regardless of battery, load, or which motor it is — it measures and corrects, every 50 ms. That's PID, the workhorse of all control. The rover that veered in Lesson 4 drove open-loop; give each wheel its own PID and it runs straight, because each wheel defends its own speed.

9. Push further

- A PID per wheel so the rover tracks straight without trimming.

- Anti-windup: clamp the integral so a stall doesn't cause a lurch.
- Tune by eye — P, then I, then D — which is far easier when you can *see* the curves. That's Lesson 8.